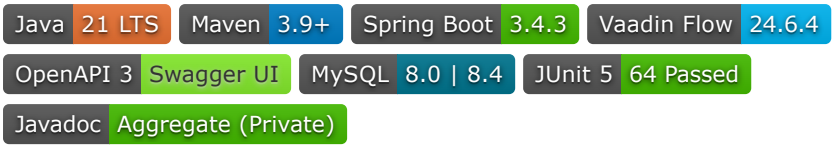
Albero di Regressione (Java)



Sistema distribuito in Java per l'apprendimento induttivo, la serializzazione binaria e la predizione supervisionata mediante **Alberi di Regressione** (Regression Trees). Il sistema integra un'architettura duale: 1. **Modalità Socket TCP Concorrente (Console):** demone multithread su porta `8080` con client terminale interattivo. 2. **Modalità Web Enterprise (REST & Vaadin):** backend Spring Boot 3.4.3 con REST API stateless, documentazione interattiva OpenAPI/Swagger UI su porta `8080` (`http://localhost:8080/swagger-ui.html`) e interfaccia web reattiva in puro Java con Vaadin Flow su porta `8081` (`http://localhost:8081`).

Il core matematico include persistenza relazionale su database MySQL (`MapDB`), calcolo ottimizzato della varianza (SSE), partizionamento in-place con quicksort e supporto ad attributi discreti e continui.

📑 Indice dei Contenuti

- Panoramica del Progetto
- ⚡ Esecuzione Rapida con Script Batch (.bat / .sh)
- Architettura del Sistema
- Indice della Documentazione
- Guida Rapida all'Avvio
  - 1. Configurazione Rapida Database MySQL
  - 2. Compilazione del Progetto
  - 3. Modalità Web: Avvio Server REST e Client Vaadin
  - 4. Modalità Socket TCP (Console Tradizionale)
  - 5. Esecuzione dei Test Automatizzati
- Risoluzione Automatica delle Dipendenze con Maven
- Generazione della Documentazione (PDF e Javadoc)
- Struttura della Repository
- Autore e Licenza

Panoramica del Progetto

Il software affronta problemi di regressione numerica supervisionata: - **Induzione Ricorsiva dell'Albero:** Minimizzazione progressiva della Somma dei Quadrati degli Errori (SSE) sui nodi interni. - **Split Polimorfici:** - *Attributi Discreti (`DiscreteNode`):* partizionamento multi-ramo con scansione lineare post-

ordinamento contiguo. - *Attributi Continui (`ContinuousNode`):* ricerca della soglia di taglio binario ottima τ minimizzando la somma degli SSE dei due rami candidati ($\leq \tau$ e $> \tau$). - **Navigazione Stateless:** Metodo `navigate(List<Integer> path)` che consente l'ispezione dell'albero e la predizione passo-passo via HTTP/REST senza dipendenza da connessioni socket aperte. - **Interfaccia Web Vaadin Flow:** Dashboard in puro Java su porta `8081` con navigazione visuale dei bivi decisionali e visualizzazione istantanea delle regole IF-THEN. - **Specifiche OpenAPI 3 e Swagger UI:** Endpoint documentati interattivamente su `/swagger-ui.html`. - **Ottimizzazione di Memoria:** Ordinamento in-place mediante **quicksort** con profondità stack $O(\log_2 n)$. - **Server Socket Concorrente (`MultiServer`):** Gestione multithread su socket TCP. - **Persistenza Binaria (`.dmp`):** Serializzazione del modello su disco per interrogazioni immediate a freddo.

⚡ Esecuzione Rapida con Script Batch (.bat / .sh)

Per garantire la massima ergonomia operativa e soddisfare sia il rigore accademico sia gli standard industriali, il progetto include una suite modulare di script per ambiente Windows (`.bat`) e sistemi Unix/Linux/macOS (`.sh`), affiancata dalla cartella `sql/` con gli script SQL di avvio rapido:

Script Windows	Script Unix	Descrizione
`build.bat`	`build.sh`	Compila e impacchetta entrambi i fat-JAR (`Server` e `Client`) saltando i test per la massima velocità.
`init-db.bat [-prova]`	`init-db.sh [-prova]`	Inizializza il database `MapDB` ed utente `MapUser` (`sql/01-init-schema.sql`). Con il flag `-prova`, inserisce anche i dataset dimostrativi `provaC` e `servo` (`sql/02-sample-data.sql`).
`start-server.bat`	`start-server.sh`	Avvia il Server Spring Boot REST su porta `8080` (con Swagger UI su `/swagger-ui.html`).
`start-client.bat`	`start-client.sh`	Avvia il Client Vaadin Flow su porta `8081` (apre automaticamente il browser su `http://localhost:8081`).
`run-local.bat [-prova]`	`run-local.sh [-prova]`	**Orchestratore All-in-One**: controlla i prerequisiti, compila se necessario, inizializza il DB con tabelle demo se specificato `-prova`, avvia Server e Client in console separate e apre la web UI nel browser.
`test.bat`	`test.sh`	Esegue l'intera suite di test automatizzati (64 test) con Maven a 0 fallimenti e 0 errori.

```
:: Esempio di avvio rapido all-in-one con tabelle demo:
run-local.bat -prova
```

Architettura del Sistema

Diagram Error: No diagram type detected matching given configuration for text: graph TB subgraph FrontendTier ["Frontend Tier"] CLI[Client Console - Client.Main] WEB[Vaadin Flow Web UI :8081 - Client.ClientApplication] end subgraph BackendTier ["Backend Tier"] MS[MultiServer - Socket Listener :8080] REST[Spring Boot REST API :8080 - TreeController] SWAGGER[Swagger UI / OpenAPI 3 :8080] RT[RegressionTree Engine - tree.*] MS --> RT REST --> RT REST -.-> SWAGGER end subgraph DataTier ["Data Tier"] DB[(MySQL MapDB)] FS[(Local Storage: .dmp Archives)] RT --> DB RT --> FS end CLI <-->|"TCP Socket"| MS WEB <-->|"HTTP/JSON RestClient"| REST

Indice della Documentazione

L'intera documentazione tecnica e operativa del progetto è disponibile sia in formato sorgente **Markdown (** `.md` **)** sia in formato ad alta definizione **PDF (** `docs/pdf/*.pdf` **)**, affiancata dalla documentazione API aggregata **Javadoc**:

Documento	Descrizione	Versione Markdown	Versione PD
Guida Tecnica Completa	Architettura three-tier, formulazione matematica SSE, quicksort in-place, split discreti/continui, protocollo socket, REST API e Vaadin.	docs/guida-tecnica.md	docs/pdf/guida-tec
Guida Utente e Installazione	Script SQL `MapDB`, credenziali `MapUser` / `map`, uso interfaccia Web Vaadin Flow, Swagger UI ed esecuzione CLI.	docs/guida-utente-installazione.md	docs/pdf/guida-u installazione.p
Documentazione della Suite di Test	Matrice di tracciabilità requisiti, 64 test automatizzati (JUnit 5, Mockito, Spring Boot Test E2E, RestClient) a 0 errori.	docs/documentazione-test.md	docs/pdf/documer test.pdf
Diagrammi UML e Architettura dei Package	Raccolta completa e visuale di tutti i diagrammi UML del sistema (Package, Classi per ogni package, Sequenza e Deployment).	docs/architettura-uml-oop.md	docs/pdf/architettu oop.pdf
Documentazione Javadoc	Documentazione API completa di tutti i package (inclusi metodi e campi privati).	docs/javadoc/index.html	—
README di Progetto	Panoramica sintetica, architettura,	README.md	docs/pdf/READM

Documento	Descrizione	Versione Markdown	Versione PD
	badge, comandi di build e avvio.		

Guida Rapida all'Avvio

1. Configurazione Rapida Database MySQL

Assicurarsi che il servizio MySQL sia attivo su `localhost:3306`, quindi eseguire lo script di bootstrap:

```
CREATE DATABASE IF NOT EXISTS MapDB;
CREATE USER IF NOT EXISTS 'MapUser'@'localhost' IDENTIFIED BY 'map';
GRANT ALL PRIVILEGES ON MapDB.* TO 'MapUser'@'localhost';
FLUSH PRIVILEGES;

USE MapDB;

DROP TABLE IF EXISTS provaC;
CREATE TABLE provaC (
    X VARCHAR(10) NOT NULL,
    Y DOUBLE NOT NULL,
    C DOUBLE NOT NULL
);

INSERT INTO provaC (X, Y, C) VALUES
('A', 2.0, 1.0), ('A', 2.0, 1.0), ('A', 1.0, 1.0), ('A', 2.0, 1.0),
('A', 5.0, 1.5), ('A', 5.0, 1.5), ('A', 6.0, 1.5), ('B', 6.0, 10.0),
('A', 6.0, 1.5), ('A', 6.0, 1.5), ('B', 10.0, 10.0), ('B', 5.0, 10.0),
('B', 12.0, 10.0), ('B', 14.0, 10.0), ('A', 1.0, 1.0);
```

2. Compilazione del Progetto

Dalla radice della repository:

```
mvn clean compile
```

3. Modalità Web: Avvio Server REST e Client Vaadin

3.1 Avvio Backend Spring Boot REST API (Porta 8080):

```
cd Server
mvn spring-boot:run
# Oppure dalla root: mvn -pl Server spring-boot:run
```

- **Swagger UI:** `http://localhost:8080/swagger-ui.html`

- **OpenAPI 3 Spec (JSON):** `http://localhost:8080/v3/api-docs`

3.2 Avvio Frontend Web Vaadin Flow (Porta 8081):

In un secondo terminale separato:

```
cd Client
mvn spring-boot:run
# Oppure dalla root: mvn -pl Client spring-boot:run
```

- **Interfaccia Web:** `http://localhost:8081`

4. Modalità Socket TCP (Console Tradizionale)

Avvio del Server Socket:

```
cd Server
mvn compile exec:java -Dexec.mainClass="Main"
# Oppure dalla root: mvn -pl Server compile exec:java -
        Dexec.mainClass="Main"
```

Avvio del Client Console:

In un secondo terminale:

```
cd Client
mvn compile exec:java -Dexec.mainClass="Client.Main"
# Oppure dalla root: mvn -pl Client compile exec:java -
        Dexec.mainClass="Client.Main"
```

5. Esecuzione dei Test Automatizzati

Esecuzione dell'intera suite (64 test unitari, d'integrazione, socket ed end-to-end):

```
mvn clean test
```

Generazione della Documentazione (PDF e Javadoc)

La documentazione API tecnica Javadoc aggregata di entrambi i moduli (inclusiva di tutti i dettagli privati e dei nodi package-private) può essere generata tramite Maven:

```
mvn javadoc:aggregate
```

L'albero HTML risultante è situato in `docs/javadoc/` e può essere consultato aprendo `docs/javadoc/index.html` con qualsiasi browser. Le versioni ad alta definizione pronte per la stampa e la consultazione offline sono archiviate in formato vettoriale in `docs/pdf/`.

Struttura della Repository

```
.
├── Client/                              # Modulo Client (Console CLI + Web
Vaadin Flow)
│   ├── pom.xml                          # Dipendenze (Vaadin 24.6.4, Spring
Boot Web)
│   ├── src/main/java/
│   │   ├── Client/                      # ClientApplication.java, Main.java,
MainView.java, TreeRestClientService.java
│   │   ├── dto/                         # Record DTO immutabili per REST
│   │   └── utility/                     # Keyboard.java (input da console)
│   └── src/test/java/                   # Suite di test modulo Client (20
test)
├── Server/                              # Modulo Server (Socket TCP +
Backend REST API)
│   ├── pom.xml                          # Dipendenze (Spring Boot, MySQL
Connector 8.4.0, OpenAPI)
│   ├── src/main/java/
│   │   ├── Main.java                    # Entrypoint Server Socket
(MultiServer su porta 8080)
│   │   ├── data/                        # Data, Attribute,
DiscreteAttribute, ContinuousAttribute
│   │   ├── database/                    # DbAccess, TableSchema, TableData,
Example, Column
│   │   ├── dto/                         # Record DTO immutabili
(TreeRequestDto, PredictionStepDto, ...)
│   │   ├── server/                      # ServerApplication, TreeController,
TreeService, GlobalExceptionHandler, MultiServer
│   │   └── tree/                        # RegressionTree, Node, SplitNode,
LeafNode, DiscreteNode, ContinuousNode
│   └── src/test/java/                   # Suite di test modulo Server (44
test)
├── docs/                                # Documentazione completa di
progetto
│   ├── guida-tecnica.md                 # Guida Tecnica Completa
│   ├── guida-utente-installazione.md    # Guida Utente ed Esempi
│   ├── architettura-uml-oop.md          # 11 Diagrammi UML ufficiali
│   ├── documentazione-test.md           # Matrice di tracciabilità e schede
test
│   ├── javadoc/                         # Javadoc HTML aggregato (visibilità
private completa)
│   └── pdf/                             # Versioni PDF ad alta definizione
dei documenti
│       ├── guida-tecnica.pdf
│       ├── guida-utente-installazione.pdf
│       ├── architettura-uml-oop.pdf
│       ├── documentazione-test.pdf
│       └── README.pdf
├── sql/                                 # Script SQL di inizializzazione
database
│   ├── 01-init-schema.sql               # Creazione database MapDB ed utente
```

```
MapUser
│   └── 02-sample-data.sql              # Tabelle dimostrative provaC (15
tuple) e servo (167 tuple)
├── build.bat / build.sh                # Compilazione ed impacchettamento
JAR
├── init-db.bat / init-db.sh            # Inizializzazione database locale
(con supporto opzionale a -prova)
├── start-server.bat / start-server.sh  # Avvio backend Spring Boot REST
(porta 8080)
├── start-client.bat / start-client.sh  # Avvio frontend Vaadin Flow (porta
8081 con auto-open browser)
├── run-local.bat / run-local.sh        # Orchestratore locale All-in-One
(supporta -prova)
├── test.bat / test.sh                  # Esecuzione suite automatizzata (64
test)
├── pom.xml                             # Root POM aggregator (Java 21 LTS,
javadoc aggregate 3.11.2)
└── README.md                           # Panoramica e indice principale
```

Autore e Licenza

- **Autore:** Marco Diana
- **Progetto:** Albero di Regressione (Corso di Metodi Avanzati di Programmazione)
- **Licenza:** Progetto ad uso didattico e accademico.

